From Transformer to LLMs & Agentic AI

STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

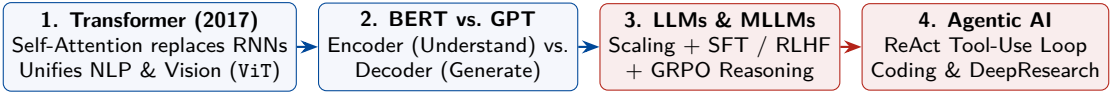
Outline & Companion Notebook

1. Part 1: The Transformer Architecture & Vision Transformers
2. Part 2: From Pre-Trained Transformers to LLMs & Frontier Agentic AI
3. Part 3: Hands-On with Hugging Face `transformers`

Hands-On Companion: `transformer_tutorial.ipynb`

- **Big-Picture Overview:** The 4-era evolution from the 2017 Transformer to Foundation Models, Multimodal LLMs, and Autonomous AI Agents.
- **Part 1 (Transformer Architecture & ViT):** Scaled Dot-Product Self-Attention ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$), Multi-Head Attention, Encoder-only (BERT) vs. Decoder-only (GPT), and Vision Transformer (ViT) image patchification.
- **Part 2 (From LLMs to Frontier Agentic AI):** Scaling laws, 3-stage post-training (Pre-training $\rightarrow$ SFT $\rightarrow$ RLHF/GRPO), Multimodal LLMs, and ReAct (`Reason -> Act -> Observe`) Coding & DeepResearch Agents.
- **Part 3 (Hands-On Hugging Face `transformers`):** `pipeline()`, subword `AutoTokenizer`, `AutoModel` embeddings, `AutoModelForSequenceClassification`, and transfer learning.

Big-Picture Overview: The 4-Era Evolution of Modern AI



How This Connects to Tutorial 10 (PyTorch)

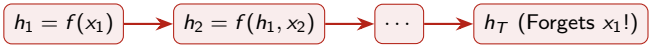
- Every frontier LLM (GPT, Claude, Gemini, DeepSeek, Qwen) is still a standard PyTorch `nn.Module`!
- Under the hood, it uses the exact same building blocks from Tutorial 10: tensor matrix multiplications (`@`), `CrossEntropyLoss`, `autograd`, and `AdamW`.

Why One Architecture Conquered All Modalities

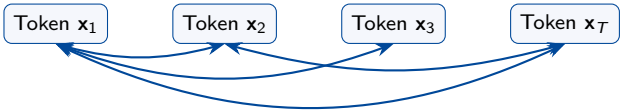
- Instead of designing separate networks for text (RNNs) and images (CNNs), the Transformer treats everything as a sequence of tokens.
- Text subwords, 16×16 image patches, audio frames, and tool calls all share one unified backbone!

Quick Design Recap: From Sequential RNNs to Parallel Self-Attention

1. Recurrent Neural Network (RNN / LSTM): $O(T)$ Sequential Bottleneck



2. Transformer Self-Attention (Vaswani et al., 2017): $O(1)$ Path Length & Full GPU Parallelism



Why RNNs Hit a Wall

- Must process tokens step-by-step ($t = 1 \to T$), preventing GPU parallelization across sequence length T.
- Long-range context gets squashed into a single fixed-size vector $\mathbf{h}_t$.

The Self-Attention Breakthrough

- Every token directly attends to every other token in 1 hop via matrix multiplication $\mathbf{QK}^\top$.
- All T tokens are processed simultaneously in parallel on GPUs!

Visual Intuition: Query ($\mathbf{Q}$), Key ($\mathbf{K}$) & Value ($\mathbf{V}$) Lookup

Information Retrieval Analogy:

- Given input token embeddings $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, project each token into three learned roles:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

- **Query ($\mathbf{q}_i$):** "What information am I looking for?"
- **Key ($\mathbf{k}_j$):** "What label/index do I advertise?"
- **Value ($\mathbf{v}_j$):** "What semantic content do I pass along if selected?"

Scaled Dot-Product Attention Formula

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)}_{\mathbf{A} \in \mathbb{R}^{T \times T} \text{ (row-stochastic)}} \mathbf{V}$$

1. **Similarity ($\mathbf{Q}\mathbf{K}^\top$):** Dot product between every query $\mathbf{q}_i$ and key $\mathbf{k}_j$.
2. **Scale ($\div\sqrt{d_k}$):** Keeps variance at 1 before softmax.
3. **Aggregate ($\mathbf{AV}$):** Weighted sum of value vectors $\sum_{j=1}^{T} A_{ij}\mathbf{v}_j$.

Why Divide by $\sqrt{d_k}$ & What Does the Attention Matrix Capture?

Two Core Insights:

- **Coreference Resolution:** In *"The animal didn't cross because **it** was tired"*, the query for `"it"` assigns 0.52 attention weight directly to `"animal"`!
- **Why $\div\sqrt{d_k}$:** If $q_{im}, k_{jm} \sim \mathcal{N}(0,1)$, then $\mathrm{Var}(\mathbf{q}_i^\top \mathbf{k}_j) = d_k$. Without dividing by $\sqrt{d_k}$, large dot products push softmax to 0/1 saturation where gradients vanish!

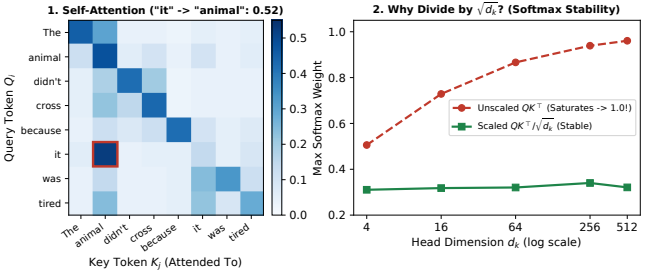


Figure Takeaway: The attention heatmap resolves long-range pronoun links (`"it"` $\rightarrow$ `"animal"` in red box), while scaling by $1/\sqrt{d_k}$ prevents softmax saturation as head dimension d_k grows.

Hands-On PyTorch: Implementing Scaled Dot-Product Attention

```python
import math, torch
import torch.nn.functional as F
def scaled_dot_product_attn(Q, K, V, mask=None):
  # Q, K, V shape:  (batch, heads, seq_len, d_k)
  d_k = Q.size(-1)
  scores = (Q @ K.transpose(-2, -1)) / math.sqrt(d_k)
  if mask is not None:
    scores = scores.masked_fill(mask == 0, -1e9)
  attn_weights = F.softmax(scores, dim=-1)
  return attn_weights @ V, attn_weights
```

Line-by-Line Shape Trace:

- `K.transpose(-2, -1)`: Swaps the last two axes from (B, H, T, d_k) to (B, H, d_k, T).
- `scores`: Batched matmul produces all-pairs (B, H, T, T) affinity matrix.
- `masked_fill(..., -1e9)`: Sets forbidden positions (padding or future tokens) to $-\infty$ so $\exp(-\infty) = 0$ after `softmax(dim=-1)`.
- **PyTorch Built-In:** `F.scaled_dot_product_attention(Q, K, V)` fuses this into FlashAttention!

Multi-Head Attention, Positional Encoding & Residual Blocks

1. Multi-Head Attention (MHA)

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$$

Running h parallel heads (each of dimension $d_k = d_{\text{model}}/h$) lets Head 1 track **syntactic verbs**, Head 2 track **pronoun coreference**, and Head 3 track **adjacent modifiers** simultaneously!

2. Positional Encoding (Sinusoidal / RoPE)

Self-Attention is permutation-equivariant (a bag of words without order!). Adding positional embeddings $\mathbf{X} + \mathbf{P}$ or Rotary Position Embedding (**RoPE**) injects token order $t \in \{1, \ldots, T\}$.

3. One Complete Transformer Block:

```
class TransformerBlock(nn.Module):
  def __init__(self, d_model=256, n_heads=8):
    super().__init__()
    self.ln1 = nn.LayerNorm(d_model)
    self.attn = nn.MultiheadAttention(
      d_model, n_heads, batch_first=True)
    self.ln2 = nn.LayerNorm(d_model)
    self.mlp = nn.Sequential(
      nn.Linear(d_model, 4 * d_model),
      nn.GELU(), nn.Linear(4 * d_model, d_model))
  def forward(self, x):  # Pre-LN residual stream
    h = x + self.attn(self.ln1(x), ...)[0]
    return h + self.mlp(self.ln2(h))
```

The Great Architecture Fork: Bidirectional vs. Causal Masking

How Attention Masking Shapes Model Families:

- **Original 2017 Transformer: Encoder–Decoder** for machine translation (e.g., T5, BART).
- **1. Encoder-Only (BERT, RoBERTa):** Uses a full bidirectional matrix—every token sees both left and right context.
- **2. Decoder-Only (GPT, Llama, DeepSeek, Qwen):** Uses a lower-triangular causal mask ($A_{ij} = 0$ for $j > i$) so token i only sees past tokens $\leq i$.

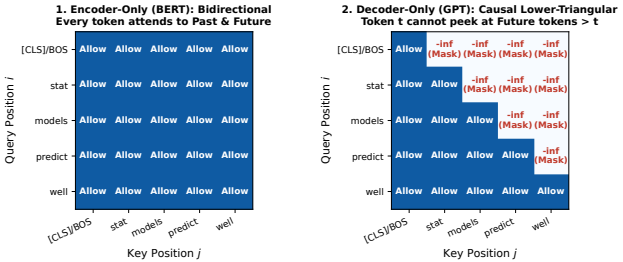


Figure Takeaway: Encoder-Only (BERT) allows full all-to-all bidirectional context for feature extraction, whereas **Decoder-Only (GPT)** masks the upper triangle ($-\infty$) for autoregressive generation.

Comparing Encoder-Only (BERT) vs. Decoder-Only (GPT) Pre-Training

Dimension	Encoder-Only (BERT, RoBERTa)	Decoder-Only (GPT, Claude, DeepSeek)
Attention Mask	Bidirectional (sees tokens $1 \dots T$)	Causal (sees past tokens $1 \dots t$ only)
Pre-Training Task	**Masked Language Modeling (MLM):**	**Next-Token Prediction (Causal LM):**
	Predict 15% masked `"The [MASK] sat"`	Maximize $\sum_{t=1}^{T} \log P_\theta(w_t \mid w_{<t})$
Token Efficiency	Loss on 15% masked positions only	Loss on all 100% of tokens in parallel!
Best Use Cases	Classification, NER, dense embeddings	Generation, reasoning, coding, agents
HF Class	`AutoModelForSequenceClassification`	`AutoModelForCausalLM`

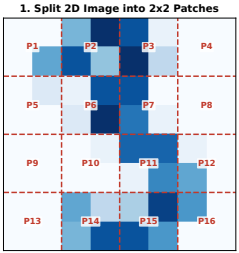
Takeaway: Why Decoder-Only Won the Frontier LLM Race

Predicting every next token $P(w_t \mid w_{<t})$ turns the entire internet into a dense supervision signal (100% token efficiency) and naturally unifies question answering, summarization, coding, and tool calling as text generation!

Vision Meets Transformer: Vision Transformer (ViT) for Images

How Do We Feed a 2D Image to a Transformer?

- Naive pixel-level attention on a 224×224 image requires $T = 50{,}176$ tokens $\Longrightarrow$ $T^2 \approx 2.5 \times 10^9$ pairs!
- **ViT Solution (Dosovitskiy et al., 2020):** An Image is Worth 16×16 Words:
  1. Split image into non-overlapping patches (e.g., $14 \times 14 = 196$ patches of size 16×16).
  2. Linearly project each patch into a token vector + positional embedding.
  3. Prepend a learnable `[CLS]` token and run a standard **Transformer Encoder**!



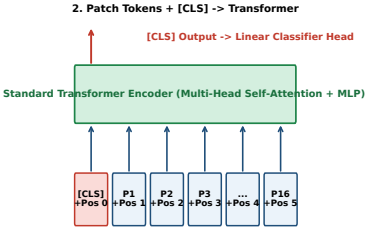


Figure Takeaway: By splitting an image into non-overlapping patch tokens (`einops.rearrange`), **Vision Transformer (ViT)** processes medical and natural images with the exact same Transformer Encoder used in NLP!

Two Scaling Paradigms: Pre-Training vs. Test-Time Reasoning

Why Scaling Changed Everything:

- **1. Pre-Training Scaling Law (Kaplan / Chinchilla):** Cross-entropy test loss follows a smooth power law $\mathcal{L}(N, D, C) \propto C^{-\alpha}$ across 7+ orders of magnitude in compute C, parameters N, and tokens D.
- **Emergent Abilities:** In-context few-shot learning, instruction following, and multi-step reasoning emerge at scale.
- **2. Test-Time Compute Scaling:** Allowing a model to generate a longer internal Chain-of-Thought (`<think>`) before answering steadily boosts math and coding accuracy!

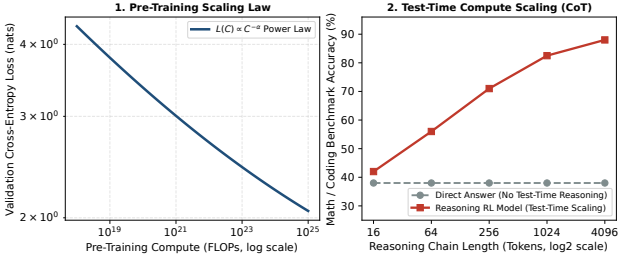
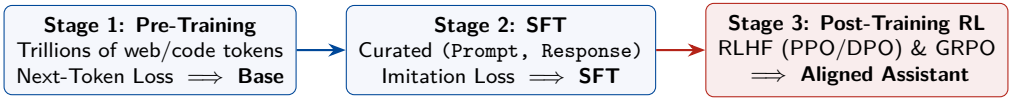


Figure Takeaway: Modern LLMs combine **Pre-Training Compute Scaling** (power-law loss reduction) with **Test-Time Compute Scaling** (spending more reasoning tokens on hard math and coding tasks).

The 3-Stage LLM Training Lifecycle: From Base Model to Assistant



Why Pre-Training Alone Is Not Enough

A raw **Base Model** only completes document prefixes: if asked *"How do I debug this Python function?"*, it may generate 5 more unanswered forum questions instead of a helpful answer!

What SFT + Post-Training RL Achieve

- **SFT** teaches the conversational `User / Assistant` turn format.
- **Post-Training RL** optimizes human preference (helpfulness, safety) and verifiable reasoning accuracy.

Stage 3 Deep Dive: Classical RLHF (PPO) vs. Reasoning RL (GRPO)

1. Classical RLHF (Reward Model + PPO)

1. Humans rank pairs of responses $(y_{\text{win}} \succ y_{\text{lose}})$ for prompt x.
2. Train a neural **Reward Model** $r_\phi(x, y)$ via Bradley–Terry loss:

$$\mathcal{L}_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_{\text{win}}) - r_\phi(x, y_{\text{lose}})\big)$$

3. Optimize LLM policy π_θ via **PPO** with a KL penalty against π_{ref}.

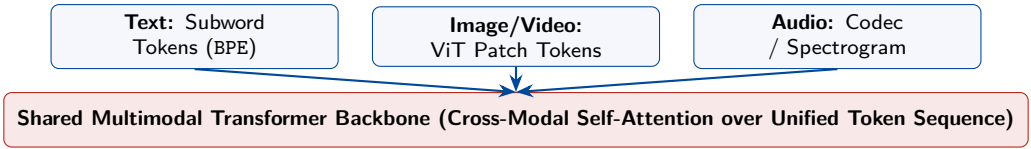
2. Frontier Reasoning RL (GRPO & Verifiers)

- Pioneered by DeepSeek-R1 / OpenAI o-series for math, logic, and code.
- **No Critic Network Needed:** Sample a group of G rollouts $\{y_1, \ldots, y_G\} \sim \pi_\theta(\cdot \mid x)$, score them with a verifiable reward (unit tests pass / exact math answer), and normalize advantage within the group:

$$\hat{A}_i = \frac{R(x, y_i) - \text{mean}(R)}{\text{std}(R) + \varepsilon}$$

- Model self-discovers verification, backtracking, and reflection!

Multimodal Foundation Models: Unified Any-to-Any Token Streams



Why Unified Tokenization Works

Once text words (BPE), medical/natural image patches (ViT), and audio frames are projected into the same d_{model} embedding space, standard Self-Attention computes cross-modal attention between words and image regions automatically!

Clinical & Scientific Applications

- Jointly reasoning over chest X-ray patches + EHR lab tables + clinical notes.
- Extracting structured tables/code directly from plots and PDFs.

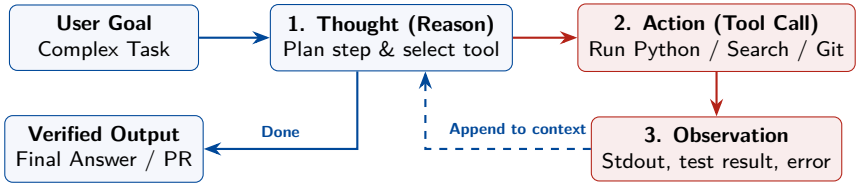
From Passive LLMs to Frontier **Agentic AI**

Capability	Passive LLM (Single-Turn Chatbot)	Autonomous LLM Agent (Agentic AI)
Loop	Single-pass Prompt $\rightarrow$ Response	Closed loop: Think $\rightarrow$ Act $\rightarrow$ Observe
Knowledge	Frozen at cutoff; can hallucinate	Queries live search, DBs, & docs
Code / Math	Guesses outputs in plain text	Runs Python, bash, & unit tests in sandbox
State	Stateless outside current prompt	Working memory & persistent workspace
Recovery	Cannot see or fix runtime bugs	Reads tracebacks and self-corrects!

Core Definition of an AI Agent

Agentic AI = Reasoning LLM + Planning & Memory + External Tools (Code/MCP) + Environment Loop

The Core Agentic Architecture: ReAct (Reason → Act → Observe)



Example ReAct Trajectory on a Data Science Task

Thought 1: I need the 5-fold CV log loss of XGBoost on `train.csv`. Let me run a Python script. → **Action 1:** `run_python("...")` → **Observation 1:** `KeyError: 'target' not found in df` → **Thought 2:** Let me inspect `df.columns` and fix the column name! → **Action 2:** `run_python("...")` → **Observation 2:** `5-fold CV log_loss = 0.4128`.

Frontier Agentic Systems: Coding Agents, MCP & DeepResearch

1. Autonomous Coding Agents

- Examples: Claude Code, Cursor, GitHub Copilot Workspace, OpenHands.
- **Workflow:** Search codebase (`grep`/`read`) $\rightarrow$ edit files $\rightarrow$ run test suite & linter $\rightarrow$ inspect failures $\rightarrow$ iterate until all checks pass and create a git commit.
- Powered by **Agentic RL**: trained end-to-end inside real Docker/Git sandboxes on software engineering tasks.

2. DeepResearch & Model Context Protocol (MCP)

- **DeepResearch Agents:** Decompose open-ended scientific questions, browse dozens of papers/sources, cross-verify citations, and synthesize technical reports.
- **Model Context Protocol (MCP):** Standardized open interface connecting LLMs to external tools, databases, and local workspaces via structured JSON schemas.

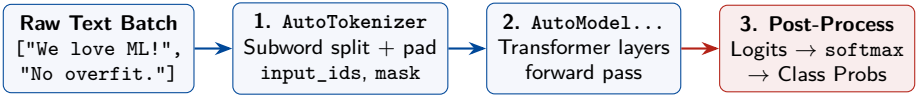
Hands-On Code: A Minimal ReAct Tool-Calling Loop in Python

```
TOOLS = {"calc": lambda expr: str(eval(expr))}
def run_react_agent(prompt, max_steps=4):
  history = [f"Question: {prompt}"]
  for step in range(max_steps):
    reply = llm_generate("\n".join(history))
    history.append(reply)
    if "Final Answer:" in reply:
      return reply
    # Parse Action: calc("...") & execute tool
    tool_name, arg = parse_action(reply)
    obs = TOOLS[tool_name](arg)
    history.append(f"Observation: {obs}")
```

Why This 12-Line Pattern Is So Powerful:

- To the Transformer, **tool use is just next-token prediction** plus string/JSON parsing!
- Whenever the model emits `Action: calc(...)`, the Python host executes the function and appends `Observation: ...` to `history`.
- The next LLM forward pass attends over the exact tool result in its context window!

Hugging Face `transformers`: 3-Stage Architecture Overview



High-Level API: `pipeline()`

Wraps all 3 stages (Tokenization → Model Forward → Softmax Post-Processing) into a single callable object for rapid prototyping.

Low-Level PyTorch API: `AutoTokenizer` + `AutoModel`

Gives full access to token IDs, attention masks, hidden-state embeddings (B, T, D), and raw logits for custom PyTorch training loops.

High-Level API: 3-Line Inference with `pipeline()`

```python
from transformers import pipeline
# 1. Load pre-trained sentiment classifier
classifier = pipeline("sentiment-analysis")
# 2. Run inference on a batch of sentences
results = classifier([
  "We are very happy to learn Transformers!",
  "We hope the model does not overfit."])
# Output: [{'label': 'POSITIVE', 'score': 0.9998},
# {'label': 'NEGATIVE', 'score': 0.8834}]
```

Popular Built-In `pipeline` Tasks:

- `"sentiment-analysis"` / `"text-classification"`: sequence-level classification.
- `"zero-shot-classification"`: classify into custom labels on the fly without retraining!
- `"feature-extraction"`: extract dense contextual embeddings.
- `"text-generation"`: autoregressive LLM decoding.

Under the Hood Step 1: Subword Tokenization & attention_mask

```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained(ckpt)
tokens = tok.tokenize("transformers rock")
batch = tok(texts, padding=True,
  truncation=True, return_tensors="pt")
```

Why Subword Tokenization (BPE / WordPiece)?

- Splits complex words into roots/suffixes (`"trans"`, `"##form"`, `"##ers"`) to eliminate `[UNK]`.
- `attention_mask`: 1 for real tokens, 0 for `[PAD]`.

WordPiece Subword Tokenization & Padding Mask for "transformers rock"

Position	0	1	2	3	4	5	6
Subwords	[CLS]	trans	##form	##ers	rock	[SEP]	[PAD]
input_ids	101	10938	14855	2545	2600	102	0
attention_mask	1	1	1	1	1	1	0

Under the Hood Step 2 & 3: `AutoModel` Embeddings & Classification Logits

Step 2: Contextual Embeddings (`AutoModel`):

```
from transformers import AutoModel
base = AutoModel.from_pretrained(ckpt)
with torch.no_grad():
  out = base(**batch)
  # shape:  (batch_size, seq_len, hidden_dim=768)
  cls_emb = out.last_hidden_state[:, 0, :]
```

- `out.last_hidden_state[:, 0, :]` extracts the 768-D sentence embedding at index 0 (`[CLS]`), ready for downstream PCA, clustering, or XGBoost/LogReg!

Step 3: Sequence Classification Logits:

```
from transformers import (
  AutoModelForSequenceClassification)
clf = AutoModelForSequenceClassification
  .from_pretrained(ckpt, num_labels=2)
out = clf(**batch, labels=y_batch)
loss = out.loss    # ready for loss.backward()!
probs = torch.softmax(out.logits, dim=-1)
```

- Passing `labels=y_batch` automatically computes cross-entropy `out.loss` for PyTorch fine-tuning!

Practical Transfer Learning: Frozen Feature Extraction vs. Fine-Tuning

```python
# Option A: Freeze backbone, train head only
for p in clf.base_model.parameters():
  p.requires_grad = False
# Option B: Full fine-tuning (small lr!)
opt = torch.optim.AdamW(
  clf.parameters(), lr=2e-5,
weight_decay=0.01)
```

- **Linear Probe (Option A):** Fast on CPU; zero risk of catastrophic forgetting on tiny datasets ($n < 100$).
- **Full Fine-Tuning (Option B):** Adapts all attention layers with a small learning rate (`lr=2e-5`) for maximum accuracy.

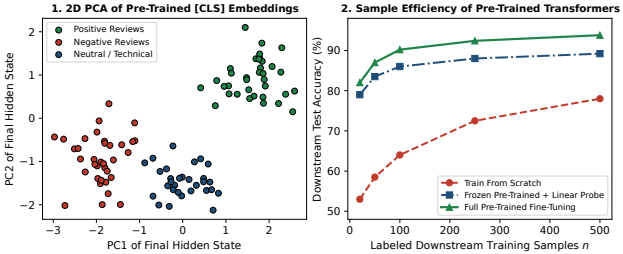


Figure Takeaway: Pre-trained `[CLS]` embeddings already separate sentiment classes in 2D PCA (left), allowing linear probing and fine-tuning to achieve $> 85\%$ accuracy with just $n = 50$ labeled examples (right).

STAT3612 Full-Course Synthesis: Choosing the Right Paradigm

Data Modality / Regime	Best Model Family	Core Inductive Bias	Library
Tabular Data (mixed cols)	GBDT / XGBoost / RF	Axis splits + ensembling	`sklearn`, `xgboost`
Margin Classification	**Regularized GLM / SVM**	L1/L2 or RKHS margin	`sklearn` (+`Scaler`)
Unlabeled Exploration	**PCA / KernelPCA / K-Means**	Low-rank var. / centroids	`sklearn`, `scipy`
2D Medical / Nat. Images	**CNN (ResNet) / ViT**	Spatial filters / patch attn	`torch`, `timm`
Text & Multimodal	Pre-Trained Transformer	Self-Attention + transfer	`transformers`
Reasoning & Tool Use	LLM Agent (ReAct + MCP)	Closed-loop verification	Agent SDKs / MCP

Key Takeaways for Modern Deep Learning & Transformers

- **Match Inductive Bias to Data:** Tree ensembles remain strong on tabular data, while Self-Attention dominates variable-length text and multimodal tokens.
- **Pre-Train Then Adapt:** Start with frozen `[CLS]` embeddings or `pipeline()` baselines before running full `AdamW` (`lr=2e-5`) fine-tuning.

Where to Go Next: Open Courses on DL, LLMs & Agentic AI

All courses below provide **free full lecture videos (YouTube)** plus **downloadable syllabi & PDF slides**:

Course	Core Topics Covered	Syllabus, Slides & Videos
Stanford CS231n (Vision)	CNNs, ResNet, Vision Transformers (ViT), VLMs	cs231n.stanford.edu
Stanford CS224N (NLP)	Self-Attention, Pre-Training, SFT, RLHF/DPO	stanford.edu/class/cs224n
Stanford CME 295 (LLMs)	RoPE, MoE, RLHF/GRPO, RAG & Agentic MCP	cme295.stanford.edu
Stanford CS336 (Scratch)	BPE, Transformer systems, scaling laws	cs336.stanford.edu
Berkeley CS294-196	Reasoning RL, ReAct, Coding Agents (`SWE-agent`)	llmagents-learning.org
Karpathy Zero to Hero	Live-coding `micrograd` (autograd) to `nanoGPT`	karpathy.ai/zero-to-hero.html

Recommended Self-Study Roadmap

Start with **Karpathy Zero to Hero** + **CS231n/CS224N** for PyTorch & Attention fundamentals $\longrightarrow$ **CME 295 / CS336** for modern LLM pre-/post-training $\longrightarrow$ **Berkeley CS294-196** for Agentic AI.